want to rely on configuring your `IOptions<T>` objects via the configuration system; you might want to configure them in code instead. The `IOptions` pattern requires only that you configure a strongly typed object before it's injected into a dependent service; it doesn't mandate that you have to bind it to an `IConfiguration` section.

TIP Technically, even if you don't configure an `IOptions<T>` at all, you can still inject it into your services. In that case, the `T` object is simply created using the default constructor.

The `Configure<T>()` method has an additional overload that takes a lambda function. The framework executes the lambda function to configure the `CurrencyOptions` object when it is injected using DI. The following listing shows an example that uses a lambda function to set the `Currencies` property on a configured `CurrencyOptions` object to a fixed array of strings.

Listing 31.12 Configuring an `IOptions` object using a lambda function

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.Configure<CurrencyOptions>(                    ❶
    builder.Configuration.GetSection("Currencies"));            ❶

builder.services.Configure<CurrencyOptions>(options =>          ❷
    options.Currencies = new string[] { "GBP", "USD"});         ❷

WebApplication app = builder.Build();
app.MapGet("/", (IOptions<CurrencyOptions> opts) => opts.Value); ❸
app.Run();
```

❶ Configures the IOptions object by binding to an IConfiguration section

❷ Configures the IOptions object by executing a lambda function

❸ The injected IOptions value is built by first binding to configuration and then applying the lambda.